

HỌC MÁY CƠ BẢN VỚI PYTHON

Chương 2 – Buổi 1

Môi trường, Python cơ bản và NumPy

(Python Basics & NumPy)

Lê Nhật Tùng

Bài giảng đi kèm giáo trình

Mục tiêu buổi học

Sau buổi học này, bạn có thể:

- Chuẩn bị môi trường lập trình trên **Google Colab** hoặc trên máy cá nhân.
- Đọc và viết các cú pháp Python cơ bản: **biến** (*variable*), **danh sách** (*list*), **hàm** (*function*), điều kiện, vòng lặp, **từ điển** (*dictionary*).
- Tạo và thao tác với **mảng NumPy** (*NumPy array*): `shape`, chỉ số, cắt lát, phép toán theo phần tử, **broadcasting** (*lan truyền kích thước*).
- Tính thống kê theo trục (`axis`), lọc bằng **mặt nạ** (*boolean mask*), tính khoảng cách kNN bằng NumPy.

Nội dung

1. Bộ công cụ và môi trường

2. Ôn nhanh Python

3. NumPy: tính toán với mảng

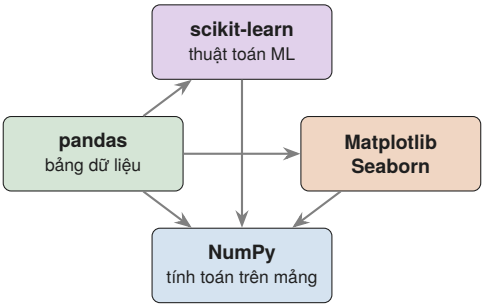
4. Tổng kết Buổi 2.1

Phần 1

Bộ công cụ và môi trường

Bộ công cụ của người làm học máy

Python trở thành ngôn ngữ số một của học máy không phải vì bản thân nó nhanh, mà nhờ một **hệ sinh thái thư viện** (*library ecosystem*) rất mạnh – mỗi thư viện lo một việc.



nền tảng: mọi thư viện đều dùng mảng NumPy

- **NumPy** (*Numerical Python*): mảng số nhiều chiều (*n-dimensional array*), tính toán nhanh – "nền móng" của mọi thư viện còn lại.
- **pandas**: dữ liệu dạng bảng có tên cột, giống Excel nhưng điều khiển bằng mã lệnh.
- **Matplotlib**, **Seaborn**: **trực quan hóa** (*data visualization*). Seaborn xây trên Matplotlib, vẽ biểu đồ thống kê đẹp chỉ với một dòng.
- **scikit-learn**: thuật toán học máy, công cụ **tiền xử lý** (*preprocessing*) và **đánh giá mô hình** (*model evaluation*).

Ví dụ đời thường: hộp đồ nghề của người thợ

Ví dụ đời thường: Không ai tự rèn cờ lê mỗi lần sửa xe

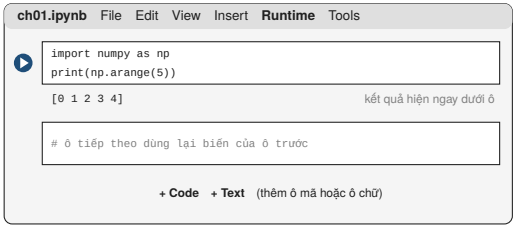
Người thợ sửa xe có sẵn hộp đồ nghề: cờ lê vặn ốc, kìm kẹp dây, tua vít mở vít. Người làm học máy cũng không tự viết từ đầu chương trình đọc file Excel hay vẽ biểu đồ – ta dùng các **thư viện** (*libraries*) do cộng đồng viết sẵn, miễn phí, đã được hàng triệu người dùng thử.

- Việc của ta: biết **dụng cụ nào dùng cho việc gì**.
- Chương trình Iris ở Chương 1 đã dùng đủ bốn nhóm: pandas giữ bảng dữ liệu, Matplotlib vẽ biểu đồ, `scikit-learn` huấn luyện kNN, và bên dưới là các mảng NumPy.
- Chương này học kỹ từng "dụng cụ".

Cách 1: Google Colab (khuyến khích cho người mới)

Google Colab là dịch vụ miễn phí cho phép viết và chạy Python ngay trên trình duyệt, **không cần cài đặt**; các thư viện dùng trong môn học đã có sẵn.

1. Vào `https://colab.research.google.com`, đăng nhập tài khoản Google.
2. **File** → **New notebook** để tạo **sổ tay** (*notebook*) mới; hoặc **Upload notebook** để mở file `.ipynb` của giáo trình.
3. Gõ mã vào **ô mã** (*code cell*), nhấn **Shift + Enter** để chạy; kết quả hiện ngay dưới ô.
4. Biến tạo ở ô trước dùng được ở ô sau (nếu ô trước đã chạy). Lỗi lạ? **Runtime** → **Restart and run all**.



Giao diện một notebook trên Colab (minh họa)

Notebook trên Colab: một vài điều cần biết

- Mã chạy trên **máy chủ của Google**; trình duyệt chỉ hiển thị kết quả $\Rightarrow$ học được cả trên máy cấu hình yếu.
- Một **notebook** (*sổ tay*) gồm nhiều ô: **ô mã** (*code cell*) chứa lệnh Python, **ô chữ** (*text cell*) chứa ghi chú – vừa viết mã vừa ghi chép như cuốn vở.
- Biến tạo ở ô trước dùng được ở ô sau **nếu ô trước đã chạy**.

Lưu ý

Colab tự xóa biến và file tạm sau khoảng 90 phút không hoạt động hoặc sau 12 tiếng. Khi quay lại, hãy **chạy lại các ô từ đầu** (**Runtime** $\rightarrow$ **Run all**). Notebook vẫn được lưu trong Google Drive.

Cách 2: Cài đặt trên máy cá nhân

1. Cài Python (phiên bản $\geq$ 3.10) từ `https://www.python.org`. Trên Windows, nhớ đánh dấu ô **Add Python to PATH**.
2. Mở cửa sổ dòng lệnh (*Command Prompt, PowerShell, Terminal*) và cài các thư viện bằng **trình quản lý gói** (*package manager*) `pip`:

```
pip install numpy pandas matplotlib seaborn scikit-learn jupyter
```

3. Cài **Visual Studio Code** cùng tiện ích mở rộng (*extension*) *Python* và *Jupyter*. Khi đó có thể mở trực tiếp file `.ipynb`, hoặc mở file `.py` đi kèm sách và chạy từng ô `# %%`.

Mẹo nhỏ

Colab phù hợp để học và thực hành nhanh; cài trên máy phù hợp khi làm dự án lớn, dữ liệu riêng tư hoặc cần làm việc không có Internet.

Kiểm tra môi trường

ch02.py — ô kiem_tra_phien_ban

```python
import sys
import numpy as np
import pandas as pd
import matplotlib
import sklearn
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 85)
print("Python      :", sys.version.split()[0])
print("NumPy       :", np.__version__)
print("pandas      :", pd.__version__)
print("Matplotlib  :", matplotlib.__version__)
print("scikit-learn:", sklearn.__version__)
```

Kết quả

```
Python      : 3.13.16
NumPy       : 2.5.3
pandas      : 3.0.5
Matplotlib  : 3.11.2
scikit-learn: 1.9.1
```

- Mỗi thư viện có thuộc tính `__version__` cho biết **phiên bản** (*version*). Phiên bản trên máy bạn có thể khác.
- Mã trong môn học chạy được với NumPy $\geq$ 1.24, pandas $\geq$ 2.0, scikit-learn $\geq$ 1.3.
- Hai dòng `pd.set_option(...)` chỉ để pandas in bảng đầy đủ cột, gọn trong khổ giấy; có thể bỏ khi làm trên Colab.

Phần 2

Ôn nhanh Python

Biến và kiểu dữ liệu

ch02.py — ô py_bien

```
ten = "An"              # chuỗi ký tự (str)
tuoi = 20               # số nguyên (int)
diem_tb = 7.85          # số thực (float)
da_tot_nghiep = False # đúng/sai (bool)
print(ten, tuoi, diem_tb, da_tot_nghiep)
print(type(ten), type(tuoi), type(diem_tb),
    ↪ type(da_tot_nghiep))
print("Tuổi sau 3 năm:", tuoi + 3)
print("Điểm làm tròn :", round(diem_tb, 1))
```

Kết quả

```
An 20 7.85 False
<class 'str'> <class 'int'> <class 'float'> <class 'bool'>
Tuổi sau 3 năm: 23
Điểm làm tròn : 7.8
```

- **Biến** (*variable*): một cái tên gắn với một giá trị – như dán nhãn lên chiếc hộp.
- 4 kiểu cơ bản: `str` (chuỗi), `int` (số nguyên), `float` (số thực – dùng **dấu chấm** `7.85`), `bool` (`True`/`False`).
- `type()` cho biết kiểu của giá trị.

Lưu ý

`round(7.85, 1)` cho 7.8: máy lưu 7,85 thành 7,8499… – số thực trên máy tính có thể lệch ở chữ số cuối.

Danh sách (list): chỉ số và cắt lát

ch02.py — ô py_list

```
diem = [7.5, 8.0, 6.5, 9.0, 5.5]          # danh sách (list) điểm của 5 sinh viên
print("Điểm đầu tiên:", diem[0])          # chỉ số bắt đầu từ 0
print("Điểm cuối cùng:", diem[-1])        # chỉ số âm: đếm từ cuối
print("Ba điểm đầu:", diem[0:3])          # cắt lát: từ vị trí 0 đến trước 3
print("Số sinh viên:", len(diem))
```

Kết quả

```
Điểm đầu tiên: 7.5
Điểm cuối cùng: 5.5
Ba điểm đầu: [7.5, 8.0, 6.5]
Số sinh viên: 5
```

Danh sách (*list*): dãy giá trị có thứ tự, viết trong ngoặc vuông.

Lưu ý

- **Chỉ số** (*index*) **bắt đầu từ 0**.
- Chỉ số âm đếm từ cuối: `diem[-1]` là phần tử cuối.
- **Cắt lát** (*slicing*) `diem[0:3]` lấy vị trí 0, 1, 2 – **không** lấy vị trí 3.

Quy tắc "từ `a` đến *trước* `b`" dùng ở khắp nơi: Python, NumPy, pandas.

Bài tập mẫu: đoán kết quả cắt lát

Cho `ds = [10, 20, 30, 40, 50, 60]`. Không chạy mã, hãy cho biết:

Biểu thức
`ds[2]`
`ds[-2]`
`ds[1:4]`
`ds[:3]`
`ds[3:]`

Bài tập mẫu: đoán kết quả cắt lát

Cho `ds = [10, 20, 30, 40, 50, 60]`. Không chạy mã, hãy cho biết:

Biểu thức	Kết quả	Giải thích
`ds[2]`	30	vị trí 2 (đếm từ 0)
`ds[-2]`	50	thứ hai tính từ cuối
`ds[1:4]`	`[20, 30, 40]`	vị trí 1, 2, 3 – **không** lấy 4
`ds[:3]`	`[10, 20, 30]`	trống phía trước = "từ đầu"
`ds[3:]`	`[40, 50, 60]`	trống phía sau = "đến hết"

Mẹo nhỏ

`ds[a:b]` luôn có $b - a$ phần tử: `ds[1:4]` có $4 - 1 = 3$ phần tử.

Hàm, câu lệnh điều kiện và vòng lặp

ch02.py — ô py_loop_ham

```python
def xep_loai(d):
    """Trả về xếp loại học lực từ điểm số d."""
    if d >= 8.5:
        return "Giỏi"
    elif d >= 7.0:
        return "Khá"
    elif d >= 5.0:
        return "Trung bình"
    return "Yếu"

for d in diem:
    print(f"Điểm {d:4.1f} -> {xep_loai(d)}")
```

Kết quả

```
Điểm  7.5 -> Khá
Điểm  8.0 -> Khá
Điểm  6.5 -> Trung bình
Điểm  9.0 -> Giỏi
Điểm  5.5 -> Trung bình
```

- `def`: định nghĩa **hàm** (*function*); `if/elif/else`: **câu lệnh điều kiện** (*conditional*); `for`: **vòng lặp** (*loop*).
- Python dùng **thụt lề** (*indentation*) (4 dấu cách) để xác định khối lệnh – thụt sai là lỗi.
- `f"Điểm {d:4.1f}"` là **f-string** (*formatted string*): chèn giá trị `d` vào chỗ ngoặc nhọn; `4.1f` = số thực, rộng 4 ký tự, 1 chữ số thập phân. Ta dùng f-string để in kết quả suốt giáo trình.

Đọc hàm `xep_loai` từ trên xuống

- `def`: định nghĩa hàm – một đoạn mã có tên, nhận đầu vào, `return` trả kết quả.
- Nếu `d >= 8.5` $\Rightarrow$ trả về "Giỏi" và **kết thúc hàm ngay**.
- Nếu không, xét `elif` (*else if* – "nếu không thì nếu"): khi đến `elif d >= 7.0` ta đã biết chắc `d < 8.5`.
- Python dùng **thụt lề** (*indentation*) 4 dấu cách để biết dòng nào thuộc `if`, `for`, `def`; thụt sai $\Rightarrow$ `IndentationError`.
- **f-string** (*formatted string*) `f"Điểm {d:4.1f}"`: chèn `d` vào chuỗi, số thực rộng 4 ký tự, 1 chữ số thập phân.

Bài tập mẫu: hàm tính tiền điện bậc thang

Ba bậc (nghìn đồng/kWh): 50 kWh đầu giá 1,8; kWh 51–100 giá 1,9; từ kWh 101 giá 2,2.

Bài tập mẫu: Tính tay

- 30 kWh: $30 \times 1{,}8 = 54$.
- 80 kWh:
  $50 \times 1{,}8 + 30 \times 1{,}9 = 90 + 57 = 147$.
- 120 kWh: $90 + 95 + 20 \times 2{,}2 = 229$.

Mỗi bậc là một nhánh `if`/`elif`/`else`. Bài toán có **công thức rõ ràng** ⇒ lập trình trực tiếp, không dùng học máy.

ch02.py — ô py_tien_dien

```python
def tien_dien(kwh):
    """Tiền điện (nghìn đồng) theo 3 bậc: 50 kWh đầu 1,8; 50 kWh tiếp 1,9;
    phần còn lại 2,2."""
    if kwh <= 50:
        return kwh * 1.8
    elif kwh <= 100:
        return 50 * 1.8 + (kwh - 50) * 1.9
    else:
        return 50 * 1.8 + 50 * 1.9 + (kwh - 100) * 2.2

for so_dien in [30, 80, 120]:
    print(f"{so_dien:4d} kWh -> {tien_dien(so_dien):6.1f} nghìn đồng")
```

Kết quả

```
  30 kWh ->   54.0 nghìn đồng
  80 kWh ->  147.0 nghìn đồng
 120 kWh ->  229.0 nghìn đồng
```

Từ điển (dict) và list comprehension

ch02.py — ô py_dict

```
1 sinh_vien = {"ten": "An", "tuoi": 20, "diem": 8.0}   #
    ↪ từ điển (dict): khóa -> giá trị
2 print(sinh_vien["ten"], "có điểm", sinh_vien["diem"])
3 sinh_vien["lop"] = "DS01"                             #
    ↪ thêm một cặp khóa-giá trị
4 print(sinh_vien)
```

Kết quả

```
An có điểm 8.0
{'ten': 'An', 'tuoi': 20, 'diem': 8.0, 'lop': 'DS01'}
```

Từ điển (*dictionary, dict*): tập các cặp **khóa – giá trị** (*key – value*); tra cứu theo khóa thay vì theo vị trí.

ch02.py — ô py_list_comp

```
1 binh_phuong = [x ** 2 for x in range(1, 6)]           #
    ↪ list comprehension
2 dat = [d for d in diem if d >= 7.0]                   #
    ↪ lọc điểm từ 7 trở lên
3 print(binh_phuong)
4 print(dat)
```

Kết quả

```
[1, 4, 9, 16, 25]
[7.5, 8.0, 9.0]
```

List comprehension: cách viết gọn để tạo danh sách mới. Đọc dòng 2: "lấy `d` cho mỗi `d` trong `diem`, *nếu* `d >= 7.0`".

Tự kiểm tra nhanh: Python cơ bản

1. `type(3.0)` trả về gì? Còn `type("3")`?
2. Với `diem = [7.5, 8.0, 6.5, 9.0, 5.5]`: `diem[1:3]` và `diem[-1]` là gì?
3. Viết list comprehension lấy các điểm **dưới** 7 trong `diem`.

Tự kiểm tra nhanh: Python cơ bản

1. `type(3.0)` trả về gì? Còn `type("3")`?
2. Với `diem = [7.5, 8.0, 6.5, 9.0, 5.5]`: `diem[1:3]` và `diem[-1]` là gì?
3. Viết list comprehension lấy các điểm **dưới** 7 trong `diem`.

Đáp án

(1) `float`; `str`. (2) `[8.0, 6.5]` và `5.5`. (3) `[d for d in diem if d < 7]` → `[6.5, 5.5]`.

Phần 3

NumPy: tính toán với mảng

Vì sao cần NumPy?

- Trong học máy, dữ liệu luôn là **các bảng số**: ma trận đặc trưng X kích thước $n \times d$, vector nhãn y gồm n số.
- NumPy cung cấp kiểu dữ liệu **ndarray** (*n-dimensional array – mảng n chiều*) để lưu và tính toán trên các bảng số này.
- So với `list` của Python, mảng NumPy:
  - chứa các phần tử **cùng kiểu** (*dtype*), lưu liền nhau trong bộ nhớ;
  - tính toán bằng mã C đã tối ưu $\Rightarrow$ **nhanh hơn hàng chục, hàng trăm lần**;
  - cho phép viết công thức toán **gần như nguyên văn**, không cần vòng lặp.

Quy ước nạp thư viện: `import numpy as np` – mọi tài liệu đều dùng tên viết tắt `np`.

List và mảng NumPy: cùng phép toán, khác ý nghĩa

ch02.py — ô np_vi_sao

```
ds = [1, 2, 3]
mang = np.array([1, 2, 3])
print("list  * 2 =", ds * 2)        # list: lặp lại danh sách hai lần!
print("mảng  * 2 =", mang * 2)      # mảng NumPy: nhân từng phần tử với 2
print("list  + [10, 20, 30] =", ds + [10, 20, 30])     # list: nối hai danh sách
print("mảng + [10, 20, 30] =", mang + np.array([10, 20, 30]))   # mảng: cộng từng cặp
```

Kết quả

```
list  * 2 = [1, 2, 3, 1, 2, 3]
mảng  * 2 = [2 4 6]
list  + [10, 20, 30] = [1, 2, 3, 10, 20, 30]
mảng + [10, 20, 30] = [11 22 33]
```

- List: `* 2` = lặp lại hai lần; `+` = nối.
- Mảng: `* 2` = nhân từng phần tử; `+` = cộng từng cặp – đúng như toán học.

Ví dụ đời thường: Máy tính bỏ túi và Excel

List như máy tính bỏ túi: tăng giá 1 000 mặt hàng phải bấm 1 000 lần. NumPy như Excel: viết công thức một lần cho cả cột.

Tạo mảng và kích thước (shape)

ch02.py — ô np_tao_mang

```
1  a = np.array([1, 2, 3, 4])                    # mảng 1
   ↪  chiều từ list
2  B = np.array([[1, 2, 3],
3                [4, 5, 6]])                    # mảng 2
                 ↪  chiều (ma trận 2 x 3)
4  print("a =", a)
5  print("B =\n", B)
6  print("a.shape =", a.shape, "  B.shape =", B.shape)
7  print("Kiểu dữ liệu của B:", B.dtype)
```

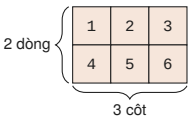
Kết quả

```
a = [1 2 3 4]
B =
 [[1 2 3]
 [4 5 6]]
a.shape = (4,)   B.shape = (2, 3)
Kiểu dữ liệu của B: int64
```

a: shape (4,)



B: shape (2, 3)



- `.shape`: **kích thước** (*shape*) – a có 4 phần tử nên `(4,)`; B có 2 dòng, 3 cột nên `(2, 3)`.
- Thói quen **in shape** trước khi tính toán giúp tránh rất nhiều lỗi.

Các hàm tạo mảng nhanh

ch02.py — ô np_ham_tao

```
print(np.zeros(4))                  # 4 số 0
print(np.ones((2, 3)))              # ma trận 2 x 3 toàn số 1
print(np.arange(0, 10, 2))          # từ 0 đến trước 10, bước 2
print(np.linspace(0, 1, 5))         # 5 số cách đều từ 0 đến 1
```

Kết quả

```
[0. 0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
```

- `np.zeros`, `np.ones`: mảng toàn số 0, toàn số 1 với kích thước cho trước.
- `np.arange(a, b, buoc)`: từ `a` đến *trước* `b`, cách nhau `buoc` (giống `range`).
- `np.linspace(a, b, n)`: `n` số **cách đều** (*evenly spaced*), *có cả* hai đầu `a` và `b` – hay dùng khi vẽ đồ thị hàm số.

Chỉ số và cắt lát: `M[dòng, cột]`

ch02.py — ô np_chi_so

```
M = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])
print("M[0, 1]   =", M[0, 1])         # dòng 0, cột 1
print("M[1]      =", M[1])            # cả dòng 1
print("M[:, 2]   =", M[:, 2])         # cả cột 2 (dấu
    ↪ : nghĩa là "tất cả")
print("M[0:2, 1:] =\n", M[0:2, 1:])   # dòng 0-1, cột
    ↪ 1 đến hết
```

Kết quả

```
M[0, 1]   = 20
M[1]      = [40 50 60]
M[:, 2]   = [30 60 90]
M[0:2, 1:] =
 [[20 30]
 [50 60]]
```

M[0, 1]

10	**20**	30
40	50	60
70	80	90

M[:, 2]

10	20	**30**
40	50	**60**
70	80	**90**

M[0:2, 1:]

10	**20**	**30**
40	**50**	**60**
70	80	90

Ô tô màu là phần được lấy ra.

- Mảng hai chiều: `M[dòng, cột]`; dấu `:` đứng một mình = “lấy tất cả”.
- `M[:, 2]`: mọi dòng của cột 2 – cách lấy **một đặc trưng** (một cột) của X.

Bài tập mẫu: chỉ số trên ma trận `M`

$$M = \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{pmatrix}$$

Biểu thức
`M[2, 0]`
`M[-1]`
`M[1:, :2]`
`M[:, 1].sum()`

Bài tập mẫu: chỉ số trên ma trận `M`

$$M = \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{pmatrix}$$

Biểu thức	Kết quả
`M[2, 0]`	70 (dòng 2, cột 0)
`M[-1]`	`[70, 80, 90]` (dòng cuối)
`M[1:, :2]`	`[[40, 50], [70, 80]]` (ma trận 2×2)
`M[:, 1].sum()`	$20 + 50 + 80 = 150$

Mẹo nhỏ

Quy tắc đọc: phần **trước** dấu phẩy chọn **dòng**, phần **sau** dấu phẩy chọn **cột**.

Phép toán theo từng phần tử

ch02.py — ô np_phep_toan

```
x = np.array([1, 2, 3])
y = np.array([10, 20, 30])
print("x + y  =", x + y)              # cộng từng phần tử
print("x * y  =", x * y)              # nhân từng phần tử
print("x ** 2 =", x ** 2)             # bình phương từng phần tử
print("x * 10 =", x * 10)             # nhân mọi phần tử với 10
```

Kết quả

```
x + y  = [11 22 33]
x * y  = [10 40 90]
x ** 2 = [1 4 9]
x * 10 = [10 20 30]
```

- Khác với `list`, phép toán trên mảng NumPy được thực hiện **theo từng phần tử** (*element-wise*).
- `a + b`: cộng từng cặp phần tử cùng vị trí; `a * 2`: nhân mọi phần tử với 2.
- Với `list`, `[1, 2] * 2` lại cho `[1, 2, 1, 2]` – **lặp** danh sách chứ không nhân!

Vector hóa: bỏ vòng lặp

ch02.py — ô np_vong_lap_vs_vector

```python
gia = [100, 250, 80, 40]
so_luong = [2, 1, 5, 10]
# Cách 1: vòng lặp Python thuần
tong = 0
for g, s in zip(gia, so_luong):
    tong += g * s
print("Tổng tiền (vòng lặp):", tong)
# Cách 2: NumPy "vector hóa" -- một dòng, nhanh hơn nhiều với dữ liệu lớn
print("Tổng tiền (NumPy)   :", np.sum(np.array(gia) * np.array(so_luong)))
```

Kết quả

```
Tổng tiền (vòng lặp): 1250
Tổng tiền (NumPy)   : 1250
```

- Hai cách cho **cùng kết quả**: vòng lặp cộng dần từng món, hoặc nhân hai mảng rồi lấy tổng trong **một dòng**.
- Cách thứ hai gọi là **vector hóa** (*vectorization*): viết công thức toán gần như nguyên văn, không cần vòng lặp.
- Với dữ liệu hàng triệu dòng, vector hóa có thể **nhanh hơn hàng chục đến hàng trăm lần**, vì phần tính toán chạy bằng mã C đã tối ưu bên trong NumPy.

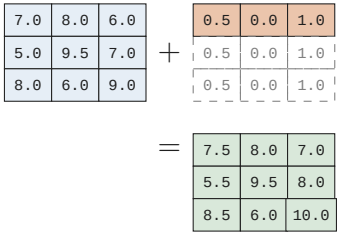
Lan truyền kích thước (broadcasting)

ch02.py — ô np_broadcasting

```
1 diem_3mon = np.array([[7.0, 8.0, 6.0],      # mỗi
  ↪ dòng: một sinh viên
2                       [5.0, 9.5, 7.0],      # mỗi
                        ↪ cột : một môn học
3                       [8.0, 6.0, 9.0]])
4 cong_them = np.array([0.5, 0.0, 1.0])       # điểm
  ↪ cộng cho từng môn
5 print(diem_3mon + cong_them)                # mảng
  ↪ 1 chiều được "lan" xuống mọi dòng
```

Kết quả

```
[[ 7.5  8.   7. ]
 [ 5.5  9.5  8. ]
 [ 8.5  6.  10. ]]
```



- **Broadcasting** (*lan truyền kích thước*): phép toán giữa hai mảng khác kích thước – NumPy tự "lan" mảng nhỏ ra cho khớp với mảng lớn (nếu được).
- Ở đây: dòng điểm cộng (3 phần tử) được lan xuống mọi dòng của bảng điểm 3×3.
- Không lan được khi kích thước không khớp, ví dụ cộng mảng 3 phần tử với mảng 4 phần tử.

Khi nào broadcasting thực hiện được?

- Trường hợp đơn giản nhất: `x * 10` – **một số** được "lan" thành `[10, 10, 10]`.
- Ma trận 3×3 + mảng **3** phần tử: mảng được lan xuống **mọi dòng** (số phần tử = **số cột**).
- Ma trận 3×3 + mảng **2** phần tử: NumPy không biết ghép phần tử nào với cột nào $\Rightarrow$ lỗi `operands could not be broadcast together`.

Sẽ gặp lại ở Chương 4

Khi **chuẩn hóa** (*standardization*) dữ liệu, ta lấy mỗi cột trừ trung bình của cột đó – tức là trừ ma trận $n \times d$ cho **một dòng** d số trung bình.

Thống kê theo trục: `axis=0` và `axis=1`

Bảng điểm: mỗi **dòng** là một sinh viên, mỗi **cột** là một môn học. Đây là điểm người mới **hay nhầm nhất**.

ch02.py — ô np_thong_ke

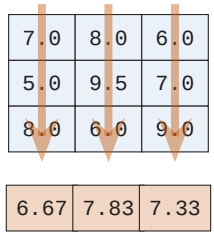
```
print("Trung bình cả bảng    :", diem_3mon.mean())
print("Trung bình mỗi môn    :", diem_3mon.mean(axis=0))   # axis=0: theo cột
print("Trung bình mỗi SV     :", diem_3mon.mean(axis=1))   # axis=1: theo dòng
print("Điểm cao nhất mỗi môn :", diem_3mon.max(axis=0))
print("Độ lệch chuẩn mỗi môn :", diem_3mon.std(axis=0).round(3))
```

Kết quả

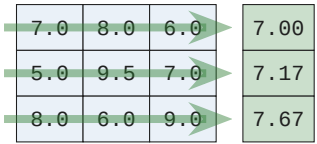
```
Trung bình cả bảng    : 7.277777777777778
Trung bình mỗi môn    : [6.66666667 7.83333333 7.33333333]
Trung bình mỗi SV     : [7.         7.16666667 7.66666667]
Điểm cao nhất mỗi môn : [8.  9.5 9. ]
Độ lệch chuẩn mỗi môn : [1.247 1.434 1.247]
```

Các hàm **thống kê** (*statistics*) `mean` (trung bình), `sum`, `min`, `max`, `std` đều nhận tham số `axis`.

Ý nghĩa của tham số `axis`



`axis=0`: gộp theo chiều dọc
$\Rightarrow$ mỗi **cột** (môn) một kết quả



`axis=1`: gộp theo chiều ngang
$\Rightarrow$ mỗi **dòng** (sinh viên) một kết quả

Mẹo nhỏ

Cách nhớ: `axis=0` làm **mất đi** chiều số 0 (chiều dòng) $\Rightarrow$ còn một giá trị cho mỗi cột. Trong học máy ta hay thống kê **theo từng đặc trưng** (từng cột), nên `axis=0` được dùng nhiều nhất.

Bài tập mẫu: tính tay trung bình theo trục

Bảng điểm 3 sinh viên (dòng) $\times$ 3 môn (cột): $\begin{pmatrix} 7{,}0 & 8{,}0 & 6{,}0 \\ 5{,}0 & 9{,}5 & 7{,}0 \\ 8{,}0 & 6{,}0 & 9{,}0 \end{pmatrix}$

Bài tập mẫu: `mean(axis=0)` **và** `mean(axis=1)`

`axis=0` – mỗi **cột** một kết quả (trung bình mỗi môn):

$$\frac{7+5+8}{3} = 6{,}667; \quad \frac{8+9{,}5+6}{3} = 7{,}833; \quad \frac{6+7+9}{3} = 7{,}333.$$

`axis=1` – mỗi **dòng** một kết quả (trung bình mỗi sinh viên):

$$\frac{7+8+6}{3} = 7; \quad \frac{5+9{,}5+7}{3} = 7{,}167; \quad \frac{8+6+9}{3} = 7{,}667.$$

Với X của Iris (150×4): `axis=0` cho **4** số, `axis=1` cho **150** số.

Lọc bằng mặt nạ True/False

ch02.py — ô np_mat_na

```
d = np.array([4.5, 7.0, 8.5, 3.0, 9.5, 6.0])
dat_mon = d >= 5                        # mảng True/False
print("d >= 5      :", dat_mon)
print("Các điểm đạt:", d[dat_mon])      # lọc bằng mặt nạ
print("Số SV đạt   :", dat_mon.sum())  # True được tính là 1
```

Kết quả

```
d >= 5      : [False  True  True False  True  True]
Các điểm đạt: [7.  8.5 9.5 6. ]
Số SV đạt   : 4
```

- So sánh một mảng với một số cho ra mảng `True`/`False` cùng kích thước, gọi là **mặt nạ** (*boolean mask*).
- Đặt mặt nạ trong ngoặc vuông ⇒ giữ lại các phần tử ứng với `True`: **lọc theo điều kiện** (*boolean indexing*).
- Vì `True` được tính là 1, `mat_na.sum()` đếm số phần tử thỏa điều kiện.
- Kỹ thuật này dùng lại **y hệt** trong pandas để lọc dòng của bảng.

Các hàm toán học của NumPy

ch02.py — ô np_ham_toan

```
v = np.array([1.0, 4.0, 9.0, 16.0])
print("căn bậc hai  :", np.sqrt(v))
print("bình phương  :", v ** 2)
print("trị tuyệt đối:", np.abs(np.array([-3, 2, -1])))
print("e mũ         :", np.exp(np.array([0, 1,
    ↪  2])).round(3))
print("logarit tự nhiên của e^2:", np.log(np.exp(2)))
```

Kết quả

```
căn bậc hai  : [1. 2. 3. 4.]
bình phương  : [  1.  16.  81. 256.]
trị tuyệt đối: [3 2 1]
e mũ         : [1.    2.718 7.389]
logarit tự nhiên của e^2: 2.0
```

Mỗi hàm tác động lên **từng phần tử**:

- `np.sqrt`: căn bậc hai – dùng trong khoảng cách (Chương 3).
- `np.abs`: trị tuyệt đối.
- `np.exp`: e^x ($e \approx 2{,}718$) – hàm sigmoid (Chương 7).
- `np.log`: logarit tự nhiên, ngược của `exp` – log loss (Chương 7).

Bài tập mẫu: tính khoảng cách kNN của Chương 1 bằng NumPy

ch02.py — ô np_khoang_cach

```python
# Tính lại bằng khoảng cách kNN của Chương 1 chỉ bằng vài dòng NumPy
X6 = np.array([[1.4, 0.2], [1.5, 0.3], [4.5, 1.5],
               [4.0, 1.2], [5.8, 2.2], [5.2, 2.0]])   # 6 bông hoa A-F
moi = np.array([4.8, 1.7])                            # bông hoa mới
chenh = X6 - moi                      # broadcasting: trừ mọi dòng cho cùng một điểm
kc = np.sqrt((chenh ** 2).sum(axis=1))   # bình phương, cộng theo dòng, lấy căn
print("Khoảng cách:", kc.round(3))
thu_tu = np.argsort(kc)               # vị trí sắp xếp từ gần đến xa
print("Thứ tự từ gần đến xa:", thu_tu)
print("3 hoa gần nhất:", [ "ABCDEF"[i] for i in thu_tu[:3] ])
```

Kết quả

```
Khoảng cách: [3.716 3.585 0.361 0.943 1.118 0.5  ]
Thứ tự từ gần đến xa: [2 5 3 4 1 0]
3 hoa gần nhất: ['C', 'F', 'D']
```

Ghép các bước tính tay với NumPy:

1. “Chênh lệch”: `X6 - moi` (broadcasting trừ **mọi** dòng).
2. “Bình phương rồi cộng hai cột”: `(chenh ** 2).sum(axis=1)`.
3. “Lấy căn”: `np.sqrt`.
4. “Sắp xếp”: `np.argsort` trả về **vị trí** theo thứ tự tăng dần.

Khớp từng chữ số với bảng tính tay: C, F, D gần nhất. Bốn dòng này là “ruột” của kNN.

Số ngẫu nhiên và hạt giống

ch02.py — ô np_ngau_nhien

```
rng = np.random.default_rng(42)        # bộ sinh số ngẫu nhiên với hạt giống 42
print(rng.integers(1, 7, size=10))     # tung xúc xắc 10 lần
print(rng.normal(loc=0, scale=1, size=5).round(3))   # 5 số theo phân phối chuẩn
```

Kết quả

```
[1 5 4 3 3 6 1 5 2 1]
[-1.302  0.128 -0.316 -0.017 -0.853]
```

- Nhiều thuật toán học máy dùng **số ngẫu nhiên** (*random numbers*): trộn dữ liệu, chia tập, khởi tạo tham số…
- Để kết quả **lặp lại được** (*reproducible*), ta cố định **hạt giống** (*random seed*).
- Cùng hạt giống 42 ⇒ máy bạn in ra **đúng những số này**; đổi hạt giống ⇒ dãy số khác.
- Trong `scikit-learn`, tham số tương ứng là `random_state` (đã gặp ở Chương 1).

Đổi hình dạng mảng: `reshape`

ch02.py — ô np_reshape

```
1 v = np.arange(1, 7)
2 print("v:", v, "shape", v.shape)
3 print("reshape(2, 3):\n", v.reshape(2, 3))
4 print("reshape(-1, 1) có shape", v.reshape(-1, 1).shape)
  ↪ # -1: tự tính
```

Kết quả

```
v: [1 2 3 4 5 6] shape (6,)
reshape(2, 3):
 [[1 2 3]
 [4 5 6]]
reshape(-1, 1) có shape (6, 1)
```

- `reshape(dòng, cột)`: **đổi hình dạng** (*reshape*) mà không đổi dữ liệu; tổng số phần tử phải giữ nguyên.
- `reshape(-1, 1)`: biến mảng một chiều thành **ma trận một cột**.
- Số `-1` nghĩa là "tự tính chiều này cho khớp".

Lưu ý

`scikit-learn` luôn yêu cầu X là mảng **hai chiều**, kể cả khi chỉ có một đặc trưng – `reshape(-1, 1)` rất hay gặp.

Ví dụ tổng hợp: dữ liệu Iris dưới dạng mảng

ch02.py — ô np_iris_tong_hop

```
1  # Ví dụ tổng hợp: dữ liệu Iris dưới dạng mảng NumPy
2  from sklearn.datasets import load_iris
3  X_iris = load_iris().data                    # mảng 150 x 4
4  print("shape:", X_iris.shape)
5  print("5 dòng đầu:\n", X_iris[:5])
6  print("Trung bình mỗi cột:", X_iris.mean(axis=0).round(2))
7  dai_canh = X_iris[:, 2]                      # cột 2: chiều dài cánh hoa
8  print("Số hoa có cánh dài hơn 5 cm:", (dai_canh > 5).sum())
9  print("Cánh dài nhất:", dai_canh.max(), " ở dòng số", dai_canh.argmax())
```

Kết quả

```
shape: (150, 4)
5 dòng đầu:
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
Trung bình mỗi cột: [5.84 3.06 3.76 1.2 ]
Số hoa có cánh dài hơn 5 cm: 42
Cánh dài nhất: 6.9  ở dòng số 118
```

Dùng gần như mọi kỹ thuật của buổi:

- `.shape` (dòng 4), cắt lát 5 dòng đầu (dòng 5);
- trung bình theo cột `axis=0` (dòng 6);
- lấy một cột `[:, 2]` (dòng 7);
- đếm bằng mặt nạ (dòng 8);
- `argmax`: **vị trí** giá trị lớn nhất (dòng 9).

42 bông hoa có cánh dài hơn 5 cm; cánh dài nhất 6,9 cm ở dòng 118.

Tự kiểm tra nhanh: NumPy

1. `np.array([[1, 2], [3, 4], [5, 6]]).shape` là gì?
2. Với X của Iris, `X.mean(axis=1)` cho bao nhiêu số? Mỗi số có ý nghĩa gì?
3. `np.array([1, 2, 3]) * np.array([2, 2, 2])` và `[1, 2, 3] * 2` cho kết quả gì?
4. Viết lệnh đếm số bông hoa có chiều rộng cánh hoa (cột 3) nhỏ hơn 1 cm.

Tự kiểm tra nhanh: NumPy

1. `np.array([[1, 2], [3, 4], [5, 6]]).shape` là gì?
2. Với X của Iris, `X.mean(axis=1)` cho bao nhiêu số? Mỗi số có ý nghĩa gì?
3. `np.array([1, 2, 3]) * np.array([2, 2, 2])` và `[1, 2, 3] * 2` cho kết quả gì?
4. Viết lệnh đếm số bông hoa có chiều rộng cánh hoa (cột 3) nhỏ hơn 1 cm.

Đáp án

(1) `(3, 2)`. (2) 150 số – trung bình 4 số đo của mỗi bông (ít ý nghĩa). (3) `[2 4 6]` và `[1, 2, 3, 1, 2, 3]`. (4) `(X[:, 3] < 1).sum()`.

Bảng thuật ngữ Việt – Anh của buổi học

Tiếng Việt	Tiếng Anh
Thư viện	Library
Sổ tay	Notebook
Ô mã	Code cell
Biến	Variable
Kiểu dữ liệu	Data type
Danh sách	List
Chỉ số	Index
Cắt lát	Slicing
Hàm	Function
Vòng lặp	Loop

Tiếng Việt	Tiếng Anh
Từ điển	Dictionary
Mảng nhiều chiều	n-dimensional array
Kích thước	Shape
Phép toán theo phần tử	Element-wise operation
Vector hóa	Vectorization
Lan truyền kích thước	Broadcasting
Trục	Axis
Mặt nạ	Boolean mask
Hạt giống ngẫu nhiên	Random seed
Đổi hình dạng	Reshape

Phần 4

Tổng kết Buổi 2.1

Tóm tắt Buổi 2.1

- Bốn nhóm thư viện: NumPy (mảng số), pandas (bảng), Matplotlib/Seaborn (biểu đồ), scikit-learn (học máy). Colab chạy Python ngay trên trình duyệt.
- Python: chỉ số bắt đầu từ 0; `[a:b]` lấy từ `a` đến **trước** `b`; hàm định nghĩa bằng `def`; thụt lề quyết định khối lệnh.
- NumPy: luôn kiểm tra `.shape`; `M[dòng, cột]`; phép toán theo từng phần tử; broadcasting lan một dòng xuống mọi dòng.
- `axis=0`: mỗi cột một kết quả; `axis=1`: mỗi dòng một kết quả.
- Mặt nạ True/False để lọc và đếm; `reshape(-1, 1)` biến mảng một chiều thành một cột.

Câu hỏi thảo luận

1. Vì sao NumPy là nền tảng của pandas, Matplotlib và scikit-learn?
2. Cho `A = np.arange(12).reshape(3, 4)`. Không chạy mã, cho biết `A.shape`, `A[1, 2]`, `A[:, 0]`, `A.sum(axis=0).shape` và `A.sum(axis=1).shape`.
3. Broadcasting là gì? Cho một ví dụ broadcasting **không** thực hiện được.
4. Vì sao cần cố định hạt giống khi dùng số ngẫu nhiên?

Bài tập Buổi 2.1

- ⋆ **2.1**: Nhiệt độ 7 ngày `[31, 33, 30, 29, 34, 35, 32]`: trung bình, cao nhất, thấp nhất; số ngày > 32 độ; các ngày trên trung bình.
- ⋆ **2.2**: Ma trận điểm 4×3: tính tay rồi kiểm tra trung bình mỗi sinh viên, mỗi môn; ai cao nhất (`np.argmax`)?
- ⋆ **2.3**: Thêm bậc thứ tư vào `tien_dien` (từ kWh 201 giá 2,6); tính tay và kiểm tra với 250 kWh.
- ⋆ **2.4**: Với X Iris: chiều dài cánh trung bình của những bông cánh dài hơn 5 cm; vị trí bông có đài rộng nhất.
- ⋆⋆⋆ **2.5**: Không dùng vòng lặp, trừ mỗi cột của ma trận điểm cho trung bình của cột; giải thích vì sao phép trừ thực hiện được.

Chi tiết: *Bài tập Buổi 2.1*, Chương 2. Buổi sau: **Buổi 2.2 – pandas, trực quan hóa và scikit-learn**.